

Day 33 — Wed 7 Oct

Countdown: 3 days to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

🎯 Degree — AI search & logic

Source: *ComputerScience_Degree/CSD_03_AI_and_MachineLearning.md*

Key points — building a knowledge base (the 2025 Q19 answer)

Knowledge-engineering process (the steps to list):

1. **Identify the task** — what questions must the KB answer? (scope)
2. **Assemble the relevant knowledge** — knowledge acquisition from experts/sources.
3. **Decide on a vocabulary** — choose the predicates, functions, constants (the **ontology**).
4. **Encode general knowledge** — write the axioms/rules about the domain.
5. **Encode the specific problem instance** — the particular facts.
6. **Pose queries to the inference procedure** and get answers.
7. **Debug and refine** the knowledge base.

TELL and ASK: you **TELL** the KB new sentences and **ASK** it queries; the inference engine answers using what has been told.

Key challenges in building a knowledge base (the "challenges" the question demands):

Challenge	Why it is hard
Knowledge acquisition	Extracting expert knowledge is slow and error-prone (the "knowledge-acquisition bottleneck")
Representation choice	Which formalism (logic, frames, rules)? Trade-off expressiveness vs efficiency
Incompleteness & uncertainty	The world is not fully known; need default/probabilistic reasoning
Consistency	Avoiding contradictions as the KB grows
The frame problem	Representing what <i>stays the same</i> after an action, without listing everything
The qualification problem	Impossible to list every precondition of an action
Scalability & maintenance	Large KBs are hard to update and keep consistent
Common-sense knowledge	Vast, implicit, hard to formalise

Knowledge representation schemes (name and compare these)

Scheme	Idea	Strength / weakness
Logic (propositional/predicate)	Facts and rules as logical sentences	Precise, sound inference; can be inefficient
Semantic networks	Graph of nodes (concepts) and labelled edges (relations, e.g. is-a, has-part); supports inheritance	Intuitive; ambiguous semantics

Frames	Objects with slots (attributes) and fillers; slots can have defaults; supports inheritance	Structured, OOP-like; the basis of many expert systems
Scripts	Stereotyped sequences of events (e.g. the "restaurant script")	Good for understanding narratives
Production rules	IF-THEN rules + working memory + inference engine	Modular, transparent; basis of expert systems

Properties a good representation should have: representational adequacy, inferential adequacy, inferential efficiency, acquisitional efficiency.

Key points — propositional and predicate logic

	Propositional	Predicate (FOL)
Units	Atomic propositions P, Q	Predicates over objects: Man(x), Loves(x,y)
Quantifiers	none	$\forall$ (universal), $\exists$ (existential)
Expressiveness	limited	can quantify over objects

Inference rules: Modus Ponens (from P and $P \rightarrow Q$ infer Q), Modus Tollens, And-elimination, resolution. **Validity/satisfiability:** *valid* = true in all models (tautology); *satisfiable* = true in some model; *unsatisfiable* = true in none.

7.2 Resolution and refutation — worked

Resolution rule (propositional): from clauses $(A \vee B)$ and $(\neg B \vee C)$ infer $(A \vee C)$ — cancel the complementary literals B and $\neg B$.

Proof by refutation: to prove $KB \models \alpha$, add $\neg \alpha$ to the KB, convert everything to **conjunctive normal form (CNF)** clauses, and apply resolution repeatedly. If you derive the **empty clause** (a contradiction), then α is proved.

Worked example — "Did Marcus hate Caesar?" style / the classic syllogism

Prove **Mortal(Socrates)** from:

- $\forall x (\text{Man}(x) \rightarrow \text{Mortal}(x))$
- $\text{Man}(\text{Socrates})$

Step 1 — to clauses (CNF).

- (1) becomes $\neg \text{Man}(x) \vee \text{Mortal}(x)$
- (2) is $\text{Man}(\text{Socrates})$

Step 2 — negate the goal. Add $\neg \text{Mortal}(\text{Socrates})$.

Step 3 — resolve.

- Resolve $\neg \text{Man}(x) \vee \text{Mortal}(x)$ with $\text{Man}(\text{Socrates})$, unifier $\{x/\text{Socrates}\} \rightarrow \text{Mortal}(\text{Socrates})$.
- Resolve $\text{Mortal}(\text{Socrates})$ with $\neg \text{Mortal}(\text{Socrates}) \rightarrow$ **empty clause** $\square$.

Contradiction derived $\Rightarrow$ **Mortal(Socrates) is proved.** ■

Steps to convert to CNF (worth listing): eliminate $\rightarrow$ and $\leftrightarrow$; move $\neg$ inwards (De Morgan); standardise variables apart; Skolemise (remove $\exists$); drop $\forall$; distribute $\vee$ over $\wedge$; rename.

Fuzzy logic

Where probability handles *uncertainty*, **fuzzy logic** handles *vagueness*: truth values are a **degree in [0,1]**, not just 0/1. A temperature can be "0.7 hot and 0.3 warm". Components: **fuzzification** (crisp $\rightarrow$ fuzzy membership), a **rule base** (IF temperature is hot THEN fan is fast),

inference, and **defuzzification** (fuzzy → crisp output). Used in control systems (washing machines, air conditioners).

	Probability	Fuzzy logic
Handles	Uncertainty (likelihood of an event)	Vagueness (degree of membership)
Value	Chance in [0,1]	Membership degree in [0,1]
Example	"70% chance of rain"	"the day is 0.7 rainy"

LISP (LISt Processing)

- Everything is an **S-expression** in prefix (Cambridge Polish) notation: `(operator operand ...)`.
- `(+ 2 3) → 5`. `(* (+ 1 2) 4) → 12`.
- Core list functions: **car** (first element), **cdr** (rest), **cons** (prepend), **list**, **append**, **null**, **atom**.
- Functions via **defun**; recursion is the main control structure; **cond/if** for branching.

Example — factorial in LISP:

```
(defun factorial (n)
  (if (<= n 1)
      1
      (* n (factorial (- n 1)))))
(factorial 5) ; => 120
```

Example — list length:

```
(defun len (lst)
  (if (null lst) 0 (+ 1 (len (cdr lst)))))
```

PROLOG (PROgramming in LOGic)

- Program = **facts + rules**; you run it by posing **queries (goals)**.
- Fact: `parent(tom, bob)`. Rule: `grandparent(X, Z) :- parent(X, Y), parent(Y, Z)`.
(`:-` reads "if"; `,` is AND; uppercase = variable.)
- Execution = **backward chaining + unification + backtracking**.

Example — family relations:

```
parent(tom, bob).
parent(bob, ann).
parent(bob, pat).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

?- grandparent(tom, ann).    % => true
?- grandparent(tom, W).     % => W = ann ; W = pat
```

Example — factorial:

```
factorial(0, 1).
factorial(N, F) :- N > 0, N1 is N-1, factorial(N1, F1), F is N*F1.
?- factorial(5, X).    % => X = 120
```

LISP vs PROLOG — the comparison table

	LISP	PROLOG
Paradigm	Functional	Logic (declarative)
Basic unit	S-expression / list	Fact and rule (Horn clauses)
Computation	Function evaluation, recursion	Unification + backward chaining + backtracking
You specify	How to compute	What is true
Control	Explicit (recursion, cond)	Implicit (the resolution engine)
Typical use	Symbolic AI, early AI systems	Expert systems, NLP, theorem proving

12.1 Knowledge-level vs symbol-level learning (the 2025 Q14 answer)

This distinction (from Dietterich) is *named explicitly* in the syllabus and was a 10-marker in 2025.

	Knowledge-level learning	Symbol-level learning
What changes	The agent acquires new knowledge — it can now answer questions/derive conclusions it could not before	The agent reorganises/represents existing knowledge more efficiently — it learns nothing new logically, just computes faster
Effect on competence	Increases what the agent knows (extends deductive closure)	No new deductive consequences — same conclusions, reached faster
Analogy	Learning a new fact or rule	Learning a shortcut / caching / re-indexing
Example	Inductive learning from examples (genuinely new generalisation)	Explanation-based learning (speed-up learning), macro-operators, caching

One-line answer: *knowledge-level* learning adds to what the system can deduce; *symbol-level* learning only makes existing deductions cheaper. EBL is the canonical symbol-level learner; induction from data is the canonical knowledge-level learner.

🕒 Forensic — cyber crime & first responder

Source: ComputerForensic/FOR_03_CyberCrimeAndIncidentResponse.md

2.1 Packet sniffing

Concept — intuition first. A network delivers data in **packets**. A **sniffer** (packet analyser) is software (or hardware) that puts a network interface into **promiscuous mode** (wired) or **monitor mode** (Wi-Fi) so it captures **every** frame it can see, not just those addressed to it, and reassembles them into readable traffic.

- On an old **hub**, every port sees every frame → sniffing is trivial.
- On a **switch**, a host normally sees only its own traffic → the attacker must first **defeat the switch: ARP spoofing/poisoning** (redirect traffic through the attacker), **MAC flooding** (overflow the CAM table so the switch fails open and broadcasts), or **port mirroring/SPAN** (the legitimate/administrative way).

Use	Legitimate	Malicious
-----	------------	-----------

Purpose	Network troubleshooting, IDS, lawful interception, forensic capture	Stealing credentials, session tokens, PII from unencrypted traffic
Tools	Wireshark , tcpdump, tshark, Zeek	Same tools, plus attack frameworks

Forensic relevance: the examiner *uses* sniffing to capture evidence (a **PCAP** file is the network equivalent of a disk image) and *investigates* sniffing as an attack. **Defence:** end-to-end encryption (TLS/HTTPS, SSH, VPN) makes captured payloads useless; switched networks, port security, and dynamic ARP inspection raise the bar.

Exam line: encryption defeats the *confidentiality* attack of sniffing but not the *metadata* — an eavesdropper still sees who talked to whom, when, and how much (traffic analysis).

2.2 Spoofing — the family (very examinable; learn the table cold)

Spoofing = forging an identity field so traffic or a message appears to come from someone else.

Type	What is forged	How it is abused	Detection / defence
IP spoofing	Source IP address in the packet header	DoS/DDoS reflection & amplification, bypassing IP-based trust, hiding origin	Ingress/egress filtering (BCP 38), reverse-path checks; hard to get replies back (blind)
MAC spoofing	Source MAC address of the NIC	Bypass MAC filtering, impersonate a device, evade tracking	Port security, DHCP snooping, 802.1X
ARP spoofing / poisoning	The IP↔MAC mapping in victims' ARP caches	Man-in-the-middle on a LAN, session hijacking, sniffing on a switch	Static ARP, Dynamic ARP Inspection , arpwatch
DNS spoofing / cache poisoning	The name→IP answer	Redirect victims to a fake site (pharming), MITM	DNSSEC , randomised source ports/query IDs, encrypted DNS
Email spoofing	The From: / envelope sender of an email	Phishing, BEC (business email compromise), malware delivery	SPF, DKIM, DMARC (see §9)
Caller-ID / SMS-sender spoofing	The displayed phone number / sender ID	Vishing/smishing, impersonating banks	Telco-side controls; user education
Website / URL spoofing (typosquatting/homograph)	The apparent domain (look-alike, Unicode homographs)	Phishing, credential theft	Certificate inspection, brand monitoring, punycode display
GPS spoofing	The location signal	Defeat geofencing, mislead navigation	Multi-constellation checks, signal-strength anomaly detection

MCQ trap: *ARP spoofing works at Layer 2 within a broadcast domain; DNS spoofing operates on name resolution; IP spoofing forges the L3 source address.* Options routinely swap these.
Pharming = DNS-level redirection to a fake site; phishing = luring the user to click.

2.4 Malware taxonomy — the whole family in one table

Malware	Defining property	Spreads by	Forensic artifacts
---------	-------------------	------------	--------------------

Virus	Attaches to a host file/program ; needs the host to be run	User executing infected files	Modified executables, changed hashes, infected file dates
Worm	Self-replicating ; spreads without a host and without user action across networks	Network exploits, shares, email	Sudden network scanning, new listening ports, mass connections
Trojan horse	Disguised as legitimate software; does not self-replicate	User is tricked into running it	Suspicious binary, persistence keys, C2 traffic
Ransomware	Encrypts files (or locks the system) and demands payment	Phishing, RDP, exploit kits	Mass file rename/encryption, ransom notes, deleted shadow copies, encryption-tool artifacts
Rootkit	Hides its presence by subverting the OS (user-mode or kernel-mode); can hide from the live OS itself	Bundled with other malware	Discrepancies between live view and offline image; hooked APIs; why dead-box imaging matters
Bootkit	Rootkit in the MBR/VBR/UEFI — loads before the OS	Boot-chain infection	MBR/ESP anomalies (see FOR_02 §3)
Keylogger	Records keystrokes (software or hardware)	Trojan, physical device	Log file, hardware device on keyboard cable, hooking
Spyware / Adware	Covert monitoring / unwanted ads and tracking	Bundled installers	Browser hijack, tracking cookies, exfil traffic
Botnet (bot/ zombie)	Compromised host under remote C2 ; used en masse for DDoS/spam/mining	Worm/trojan	Beaconing to C2, IRC/HTTP/DNS command traffic
Logic bomb	Dormant code that fires on a condition/ date	Insider, trojan	Conditional trigger in code; ties to insider threat
Backdoor / RAT	Covert remote access channel	Trojan, exploit	Listening service, new account, remote-access tool
Fileless malware	Lives in memory / living-off-the-land (PowerShell, WMI), little on disk	Script, macro	Memory-only artifacts → volatile capture essential (Volatility)
Cryptominer	Steals CPU/GPU to mine cryptocurrency	Trojan, drive-by	High resource use, mining-pool traffic
APT	<i>Advanced Persistent Threat</i> — a well-resourced actor, long dwell time, stealthy	Targeted, multi-stage	Low-and-slow persistence, custom tooling, lateral movement

The three canonical distinctions for MCQs:

1. **Virus needs a host and user action; worm is self-replicating and self-propagating; trojan does not self-replicate at all.** These three are swapped in every question set.
2. **Rootkit = hiding**, not damage per se. Its existence is the strongest textbook argument for **dead-box (offline) imaging** and **memory forensics** — a compromised live OS cannot be trusted to report its own state.
3. **Fileless / APT** → the disk may be nearly clean; **RAM is the crime scene** → capture volatile memory first (order of volatility, §4).

§2.5 Likely exam questions (Units on cyber crime & attacks)

Marks	Question
-------	----------

5	Classify cyber crimes with examples.
5	Distinguish between a virus, a worm and a trojan horse.
5	What is packet sniffing? How is it done on a switched network?
5	Distinguish between phishing, vishing and smishing.
5	Distinguish internal and external attacks.
10	Explain the different types of spoofing with examples and their defences.
10	Describe the common ATM and UPI frauds and how each is investigated.
10	Explain SQL injection, XSS and CSRF, and how each appears in web-server evidence.
15	"Attribution is the central problem of cyber-crime investigation." Discuss with reference to spoofing, anonymisation and the classification of cyber crimes.
15	Discuss malware taxonomy and explain why rootkits and fileless malware change the forensic acquisition strategy.

MCQ traps

- **Worm self-replicates without user action; virus needs a host + execution.**
- **ARP spoofing = L2 MITM; DNS spoofing/poisoning = fake name resolution (pharming).**
- **Promiscuous mode** (wired) / **monitor mode** (Wi-Fi) enables sniffing.
- **EMV chip resists cloning; magstripe fallback is the weakness.**
- **CSRF rides the authenticated session; XSS runs injected script.**
- Scanning-a-QR-code **pays**, it does not receive money (UPI collect/QR scam).

§3. The First Responder and Incident Response

3.2 First responder — role and duties

1. **Secure and isolate the scene** — keep people away from keyboards and power; treat it like any crime scene (photograph, log entries/exits, don't let anyone "just save their work").
2. **Assess power state** — is the machine **on, off, or sleeping**? This single fact drives every subsequent decision (§3.4).
3. **Do not touch the keyboard/mouse** except as a deliberate, documented capture step.
4. **Photograph everything** — screen (as displayed), cable layout, serial numbers, connected devices, the room, and any notes/passwords stuck nearby.
5. **Identify and preserve volatile evidence** if trained and authorised (RAM, running processes, network connections) **before** power decisions.
6. **Identify all storage and networked devices** — don't miss the NAS in the cupboard, the phone charging on the desk, the cloud that this machine is logged into.
7. **Prevent remote wiping** — isolate from the network (pull the network cable / Faraday bag for phones) so nobody can trigger a remote wipe or alter data.
8. **Establish and begin the chain of custody** (see FOR_01 §4.5) — label, seal, log.
9. **Do not run programs on the suspect machine** or open files "to check" — every action changes the evidence.
10. **Hand over** to the examiner with complete notes.

3.3 First responder toolkit

Category	Items
Documentation	Notebook/forms, chain-of-custody forms, evidence labels, camera , marker pens
Anti-static & packaging	Anti-static bags, Faraday bags/boxes (for phones/radios), evidence tape, boxes, cable ties
Acquisition hardware	Hardware write blockers (SATA/IDE/USB/NVMe), forensic imaging device/duplicator , sterile (wiped, verified) target drives, adapters and connectors
Live/volatile capture	Trusted incident-response USB with statically-linked tools (memory-capture, process/network listing), a clean external drive to receive the dump
Software	Bootable forensic OS/live CD (e.g. a Linux forensic distro), imaging tools (dd/dcfldd, FTK Imager), hashing tools
Miscellaneous	Torch, screwdriver/tool kit, gloves, tamper-evident seals, spare storage, power strip, UPS/portable power to keep a live machine alive during capture

9.3 SPF, DKIM, DMARC (the anti-spoofing trio — define crisply)

Mechanism	What it checks	Passes when
SPF (Sender Policy Framework)	Is the sending server's IP authorised to send for the envelope domain? (DNS TXT record lists allowed IPs)	The connecting IP is in the domain's SPF record
DKIM (DomainKeys Identified Mail)	Was the message cryptographically signed by the domain and unaltered in transit? (public key in DNS)	The DKIM signature verifies against the domain's published key
DMARC (Domain-based Message Authentication, Reporting & Conformance)	Do SPF/DKIM align with the visible From: domain , and what policy to apply (none/quarantine/reject) if not?	Aligned SPF or DKIM pass; policy tells receivers what to do on failure

Spoof indicator summary: `spf=fail` and/or `dkim=fail` and/or `dmARC=fail`, plus a **From: domain** that differs from the authenticated/**Return-Path: domain**, plus an originating IP that does not belong to the claimed sender's network ⇒ **spoofed / phishing**.